Object Oriented Programming

DI04000021

LABORATORY MANUAL

D.E. Semester-IV

Computer Engineering

Prepared By:-
CE/IT Department
Vadodara Institute of Engineering (903)
Kotambi, Vadodara – 391510
Academic Year : 2025-2026

Vadodara Institute of Engineering (903)
Object Oriented Programming (DI0400021)

Index

Sr No	Practicals
1	Create simple programs in Java. a. Create a Java program to print "Hello World", then compile, debug and execute it using Java compiler and interpreter. b. Create a Java program to perform arithmetic operations and demonstrate type casting.
2	Create Java programs using control structures. a. Create a Java program to find maximum of three numbers. b. Create a Java program to find sum of digits of a number
3	Create Java programs using arrays. a. Create a Java program to find sum and average of array elements. b. Create a Java program to perform addition of two matrices.
4	Create a Student class with fields and methods in Java program. Demonstrate object creation and method calling.
5	Create Java programs using constructor, this and static keywords. a. Create a Java program to demonstrate default and parameterized constructors and the use of 'this' keyword. b. Create a Java program to demonstrate different uses of 'static' keyword.
6	Create a Java program to demonstrate method overloading.
7	Create a Java program to perform various String operations of using built-in String methods.
8	Create a Java program to accept input using the Scanner class, store values and demonstrate autoboxing and unboxing
9	Create a Java program to demonstrate single and multilevel inheritance. (e.g., class Person → Employee → Manager)
10	Write a Java practical to declare a 1D array of 10 integers, accept values from the user, and find the maximum, minimum, sum, and average of all elements.
11	Write a Java practical to create a Student class with fields name, roll number, and marks using a parameterized constructor and a display() method to print details of multiple student objects.
12	Write a Java practical to demonstrate inheritance and method overriding by creating a parent class Animal and a child class Dog that overrides the sound() method using the super keyword.
13	Write a Java practical to demonstrate the Collections Framework by creating an ArrayList of student names to perform add, remove, and search operations and a HashSet to store and display unique roll numbers using a for-each loop.
14	Write a Java practical to calculate the percentage of a student based on marks of 5 subjects and display the corresponding grade using if-else ladder control structure.

Vadodara Institute of Engineering (903)
Object Oriented Programming (DI0400021)

Software Requirements

Sr No	Software Requirement	Hardware Requirement
1	Visual Studio	64-bit OS
		RAM-16 GB
		Processor (Intel i5/i7, AMD Ryzen, etc.)
		Hard Disk- 512 GB

Date:

PRACTICAL - 1

Aim*:* Create simple programs in Java.

a. Create a Java program to print "Hello World", then compile, debug and execute it using Java compiler and interpreter.

b. Create a Java program to perform arithmetic operations and demonstrate type casting.

Program/Implementation:

A: Hello World Program

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello World");
   }
}
```

B: Arithmetic Operations & Type Casting

```
public class ArithmeticOperations {
   public static void main(String[] args) {

      // ---- Arithmetic Operations ----
      int a = 20;
      int b = 6;

      System.out.println("===== Arithmetic Operations =====");
      System.out.println("a = " + a + ", b = " + b);
      System.out.println("Addition       : a + b = " + (a + b));
      System.out.println("Subtraction    : a - b = " + (a - b));
      System.out.println("Multiplication : a * b = " + (a * b));
      System.out.println("Division       : a / b = " + (a / b));       // Integer division
      System.out.println("Modulus        : a % b = " + (a % b));

      // ---- Type Casting ----
      System.out.println("\n===== Type Casting =====");

      // 1. Implicit Casting (Widening): int → double (automatic)
      int intVal = 100;
      double doubleVal = intVal;  // No explicit cast needed
```

```
        System.out.println("Implicit Cast (int to double) : " + intVal + " → " + doubleVal);

        // 2. Explicit Casting (Narrowing): double → int (manual)
        double pi = 3.99;
        int truncated = (int) pi;  // Decimal part is truncated
        System.out.println("Explicit Cast (double to int) : " + pi + " → " + truncated);

        // 3. int to float
        int marks = 75;
        float percentage = (float) marks / 100 * 100;
        System.out.println("int to float cast            : " + marks + " → " + percentage + "%");

        // 4. char to int (ASCII value)
        char ch = 'A';
        int asciiVal = (int) ch;
        System.out.println("char to int (ASCII)          : '" + ch + "' → " + asciiVal);

        // 5. int to char
        int code = 90;
        char letter = (char) code;
        System.out.println("int to char                  : " + code + " → '" + letter + "'");

        // 6. Demonstrating precision in division using casting
        System.out.println("\n===== Division with Type Casting =====");
        System.out.println("Integer Division  : " + a + " / " + b + " = " + (a / b));
        System.out.println("Float Division    : " + a + " / " + b + " = " + ((float) a / b));
        System.out.println("Double Division   : " + a + " / " + b + " = " + ((double) a / b));
    }
}
```

Output:

```
===== Arithmetic Operations =====
a = 20, b = 6
Addition       : a + b = 26
Subtraction    : a - b = 14
Multiplication : a * b = 120
Division       : a / b = 3
Modulus        : a % b = 2

===== Type Casting =====
Implicit Cast (int to double) : 100 ? 100.0
Explicit Cast (double to int) : 3.99 ? 3
int to float cast             : 75 ? 75.0%
char to int (ASCII)           : 'A' ? 65
int to char                   : 90 ? 'Z'

===== Division with Type Casting =====
Integer Division  : 20 / 6 = 3
Float Division    : 20 / 6 = 3.3333333
Double Division   : 20 / 6 = 3.3333333333333335

=== Code Execution Successful ===
```

Fig 1.1 Output

Logical Applications:

1. What is the purpose of public static void main(String[] args) in Java?

_____

_____

_____

2. What is the difference between implicit and explicit type casting?

_____

_____

_____

Date:

PRACTICAL -2

Aim: Create Java programs using control structures.
a. Create a Java program to find maximum of three numbers.
b. Create a Java program to find sum of digits of a number

Program/Implementation:

A: Maximum of Three Numbers

```
import java.util.Scanner;

public class MaxOfThree {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.println("===== Find Maximum of Three Numbers =====");
      System.out.print("Enter first number  : ");
      int a = sc.nextInt();

      System.out.print("Enter second number : ");
      int b = sc.nextInt();

      System.out.print("Enter third number  : ");
      int c = sc.nextInt();

      int max;

      // Using if-else-if ladder
      if (a >= b && a >= c) {
         max = a;
      } else if (b >= a && b >= c) {
         max = b;
      } else {
         max = c;
      }

      System.out.println("\nNumbers entered : " + a + ", " + b + ", " + c);
      System.out.println("Maximum number  : " + max);

      // Also demonstrating using nested if
```

```
        System.out.println("\n--- Using Nested If ---");
        int maxNested;
        if (a > b) {
            if (a > c) {
                maxNested = a;
            } else {
                maxNested = c;
            }
        } else {
            if (b > c) {
                maxNested = b;
            } else {
                maxNested = c;
            }
        }
        System.out.println("Maximum (Nested If) : " + maxNested);

        // Also demonstrating using ternary operator
        int maxTernary = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
        System.out.println("Maximum (Ternary)   : " + maxTernary);

        sc.close();
    }
}
```

B: Sum of Digits of a Number

```
import java.util.Scanner;

public class SumOfDigits {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("===== Sum of Digits of a Number =====");
        System.out.print("Enter a number : ");
        int number = sc.nextInt();

        // Store original number for display
        int original = number;

        // Handle negative numbers
        if (number < 0) {
            number = -number;
        }
```

```
        int sum    = 0;
        int digit  = 0;
        int temp   = number;

        System.out.println("\n--- Step-by-Step Calculation ---");
        System.out.printf("%-10s %-10s %-10s%n", "Number", "Digit", "Sum");
        System.out.println("-----------------------------");

        // Using while loop to extract and sum digits
        while (temp > 0) {
            digit = temp % 10;      // Extract last digit
            sum   = sum + digit;    // Add digit to sum
            System.out.printf("%-10d %-10d %-10d%n", temp, digit, sum);
            temp  = temp / 10;      // Remove last digit
        }

        System.out.println("-----------------------------");
        System.out.println("Original Number : " + original);
        System.out.println("Sum of Digits   : " + sum);

        // ---- Bonus: Using for loop approach ----
        System.out.println("\n--- Using For Loop ---");
        int sumFor = 0;
        for (int n = number; n > 0; n /= 10) {
            sumFor += n % 10;
        }
        System.out.println("Sum of Digits (for loop) : " + sumFor);

        // ---- Bonus: Using String approach ----
        System.out.println("\n--- Using String Method ---");
        String numStr = Integer.toString(number);
        int sumStr = 0;
        for (int i = 0; i < numStr.length(); i++) {
            sumStr += Character.getNumericValue(numStr.charAt(i));
        }
        System.out.println("Sum of Digits (String)   : " + sumStr);

        sc.close();
    }
}
```

Output:

```
===== Find Maximum of Three Numbers =====
Enter first number  : 12
Enter second number : 32
Enter third number  : 34

Numbers entered : 12, 32, 34
Maximum number  : 34

--- Using Nested If ---
Maximum (Nested If) : 34
Maximum (Ternary)   : 34

=== Code Execution Successful ===
```

Fig 2.1 Output

Output

```
===== Sum of Digits of a Number =====
Enter a number : 123

--- Step-by-Step Calculation ---
Number     Digit      Sum
------------------------------
123        3          3
12         2          5
1          1          6
------------------------------
Original Number : 123
Sum of Digits   : 6

--- Using For Loop ---
Sum of Digits (for loop) : 6

--- Using String Method ---
Sum of Digits (String)   : 6
```

Fig 2.2 Output

Logical Applications:

1. What is the difference between if-else and ternary operator?

_____

_____

_____

2. How does % (modulus) operator help in extracting digits from a number?

_____

_____

_____

Date:

PRACTICAL - 3

Aim: Create Java programs using arrays.
a. Create a Java program to find sum and average of array elements.
b. Create a Java program to perform addition of two matrices.

Program/Implementation:

A: Sum and Average of Array Elements

```
import java.util.Scanner;

public class SumAndAverage {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.println("===== Sum and Average of Array Elements =====");
      System.out.print("Enter the number of elements : ");
      int n = sc.nextInt();

      int[] arr = new int[n];

      // Input array elements
      System.out.println("Enter " + n + " elements :");
      for (int i = 0; i < n; i++) {
         System.out.print("Element [" + i + "] : ");
         arr[i] = sc.nextInt();
      }

      // Display array
      System.out.println("\n--- Array Elements ---");
      System.out.print("Array : [ ");
      for (int i = 0; i < n; i++) {
         System.out.print(arr[i]);
         if (i != n - 1) System.out.print(", ");
      }
      System.out.println(" ]");

      // Calculate sum
      int sum = 0;
      System.out.println("\n--- Step-by-Step Sum Calculation ---");
      for (int i = 0; i < n; i++) {
```

```
            sum += arr[i];
            System.out.println("After adding arr[" + i + "] = " + arr[i]
                    + "  →  Running Sum = " + sum);
        }

        // Calculate average
        double average = (double) sum / n;

        // Find max and min as bonus
        int max = arr[0];
        int min = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] > max) max = arr[i];
            if (arr[i] < min) min = arr[i];
        }

        // Final Results
        System.out.println("\n========== Results ==========");
        System.out.println("Number of Elements : " + n);
        System.out.println("Sum of Elements    : " + sum);
        System.out.printf ("Average            : %.2f%n", average);
        System.out.println("Maximum Element    : " + max);
        System.out.println("Minimum Element    : " + min);
        System.out.println("============================");

        sc.close();
    }
}
```

B: Addition of Two Matrices

```
import java.util.Scanner;

public class MatrixAddition {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("===== Matrix Addition =====");
        System.out.print("Enter number of rows    : ");
        int rows = sc.nextInt();
        System.out.print("Enter number of columns : ");
        int cols = sc.nextInt();

        int[][] matA   = new int[rows][cols];
        int[][] matB   = new int[rows][cols];
```

```
int[][] result = new int[rows][cols];

// Input Matrix A
System.out.println("\nEnter elements of Matrix A (" + rows + "x" + cols + ") :");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        System.out.print("A[" + i + "][" + j + "] : ");
        matA[i][j] = sc.nextInt();
    }
}

// Input Matrix B
System.out.println("\nEnter elements of Matrix B (" + rows + "x" + cols + ") :");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        System.out.print("B[" + i + "][" + j + "] : ");
        matB[i][j] = sc.nextInt();
    }
}

// Matrix Addition
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        result[i][j] = matA[i][j] + matB[i][j];
    }
}

// Method to print a matrix neatly
System.out.println("\n========== Matrix A ==========");
printMatrix(matA, rows, cols);

System.out.println("\n========== Matrix B ==========");
printMatrix(matB, rows, cols);

System.out.println("\n====== Resultant Matrix (A + B) ======");
printMatrix(result, rows, cols);

// Show element-wise addition
System.out.println("\n--- Element-wise Addition ---");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        System.out.println("A[" + i + "][" + j + "] + B[" + i + "][" + j + "] = "
                + matA[i][j] + " + " + matB[i][j] + " = " + result[i][j]);
    }
}
```

```
        sc.close();
    }

    // Helper method to print matrix in grid format
    static void printMatrix(int[][] mat, int rows, int cols) {
        for (int i = 0; i < rows; i++) {
            System.out.print("| ");
            for (int j = 0; j < cols; j++) {
                System.out.printf("%4d ", mat[i][j]);
            }
            System.out.println("|");
        }
    }
}
```

Output:

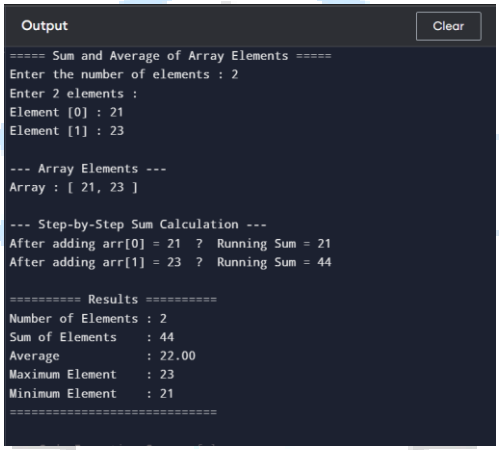

Fig 3.1 Output

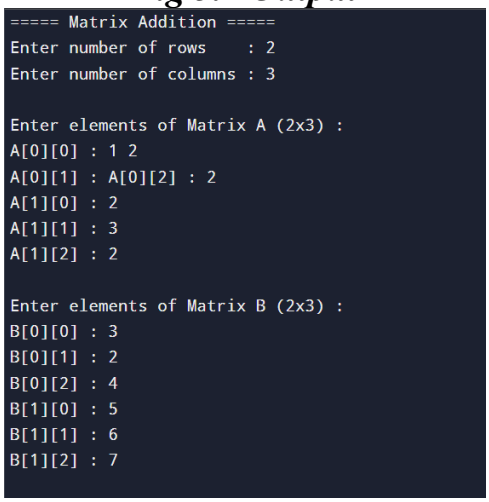

Fig 3.2 Output

```
====== Resultant Matrix (A + B) ======
|    4    4    6 |
|    7    9    9 |

--- Element-wise Addition ---
A[0][0] + B[0][0] = 1 + 3 = 4
A[0][1] + B[0][1] = 2 + 2 = 4
A[0][2] + B[0][2] = 2 + 4 = 6
A[1][0] + B[1][0] = 2 + 5 = 7
A[1][1] + B[1][1] = 3 + 6 = 9
A[1][2] + B[1][2] = 2 + 7 = 9
```

Fig 3.3 Output

Logical Applications:

1. What is the default value of an uninitialized integer array in Java?

2. What is the difference between a 1D and 2D array?

Date:

PRACTICAL - 4

Aim*:* Create a Student class with fields and methods in Java program. Demonstrate object creation and method calling.

Program/Implementation:

```
import java.util.Scanner;
public class Student {
   // ===== FIELDS (Instance Variables) =====
   int    rollNo;
   String name;
   String branch;
   int    age;
   double marks[];
   double percentage;
   String grade;

   // ===== CONSTRUCTOR (Default) =====
   Student() {
      rollNo     = 0;
      name       = "Unknown";
      branch     = "Unknown";
      age        = 0;
      marks      = new double[5];
      percentage = 0.0;
      grade      = "N/A";
   }

   // ===== CONSTRUCTOR (Parameterized) =====
   Student(int rollNo, String name, String branch, int age, double[] marks) {
      this.rollNo = rollNo;
      this.name   = name;
      this.branch = branch;
      this.age    = age;
      this.marks  = marks;
      this.percentage = calculatePercentage();
      this.grade      = calculateGrade();
   }

   // ===== METHOD: Calculate Percentage =====
   double calculatePercentage() {
```

```
        double total = 0;
        for (double m : marks) {
            total += m;
        }
        return total / marks.length;
    }

    // ===== METHOD: Calculate Grade =====
    String calculateGrade() {
        if (percentage >= 90)      return "O  (Outstanding)";
        else if (percentage >= 75) return "A+ (Excellent)";
        else if (percentage >= 60) return "A  (Very Good)";
        else if (percentage >= 50) return "B  (Good)";
        else if (percentage >= 40) return "C  (Average)";
        else                       return "F  (Fail)";
    }

    // ===== METHOD: Display Student Details =====
    void displayDetails() {

System.out.println("╔═══════════════════════════════════════════════╗");
        System.out.println("║          STUDENT DETAILS              ║");

System.out.println("╠═══════════════════════════════════════════════╣");
        System.out.printf ("║  Roll No   : %-24d║%n", rollNo);
        System.out.printf ("║  Name      : %-24s║%n", name);
        System.out.printf ("║  Branch    : %-24s║%n", branch);
        System.out.printf ("║  Age       : %-24d║%n", age);

System.out.println("╠═══════════════════════════════════════════════╣");
        System.out.println("║          MARKS DETAILS                ║");

System.out.println("╠═══════════════════════════════════════════════╣");
        String[] subjects = {"Maths ", "Science", "English", "History", "CS     "};
        for (int i = 0; i < marks.length; i++) {
            System.out.printf("║  %s    : %-24.1f║%n", subjects[i], marks[i]);
        }

System.out.println("╠═══════════════════════════════════════════════╣");
        System.out.printf ("║  Percentage: %-24.2f║%n", percentage);
        System.out.printf ("║  Grade     : %-24s║%n", grade);

System.out.println("╚═══════════════════════════════════════════════╝");
    }
```

```
    // ===== METHOD: Update Marks =====
    void updateMarks(double[] newMarks) {
        this.marks      = newMarks;
        this.percentage = calculatePercentage();
        this.grade      = calculateGrade();
        System.out.println("✓ Marks updated successfully for " + name);
    }

    // ===== METHOD: Check Pass or Fail =====
    void checkResult() {
        boolean passed = true;
        System.out.println("\n--- Result Card for " + name + " ---");
        String[] subjects = {"Maths", "Science", "English", "History", "CS"};
        for (int i = 0; i < marks.length; i++) {
            String status = (marks[i] >= 40) ? "PASS" : "FAIL";
            if (marks[i] < 40) passed = false;
            System.out.printf("%-10s : %5.1f  [%s]%n", subjects[i], marks[i], status);
        }
        System.out.println("-----------------------------");
        System.out.println("Overall Result : " + (passed ? "✓ PASSED" : "✗ FAILED"));
    }

    // ===== METHOD: Compare Two Students =====
    void compareWith(Student other) {
        System.out.println("\n--- Comparing Students ---");
        System.out.printf("%-12s : %-15s vs %-15s%n", "Name",
                          this.name, other.name);
        System.out.printf("%-12s : %-15.2f vs %-15.2f%n", "Percentage",
                          this.percentage, other.percentage);
        if (this.percentage > other.percentage) {
            System.out.println("🏆 " + this.name + " scored higher!");
        } else if (other.percentage > this.percentage) {
            System.out.println("🏆 " + other.name + " scored higher!");
        } else {
            System.out.println("🏆 Both students scored equally!");
        }
    }
    // ===== MAIN METHOD =====
    public static void main(String[] args) {
System.out.println("╔═══════════════════════════════════════════╗");
        System.out.println("║   STUDENT CLASS DEMONSTRATION       ║");

System.out.println("╚═══════════════════════════════════════════╝");
```

```
        // --- Object 1: Using Parameterized Constructor ---
        System.out.println("\n>>> Creating Student 1 using Parameterized Constructor...");
        double[] marks1 = {88.0, 92.0, 78.0, 85.0, 95.0};
        Student s1 = new Student(101, "Rahul Sharma", "Computer Science", 20, marks1);

        // --- Object 2: Using Parameterized Constructor ---
        System.out.println(">>> Creating Student 2 using Parameterized Constructor...");
        double[] marks2 = {72.0, 65.0, 80.0, 55.0, 70.0};
        Student s2 = new Student(102, "Priya Patel", "Information Tech", 21, marks2);

        // --- Object 3: Using Default Constructor then setting values ---
        System.out.println(">>> Creating Student 3 using Default Constructor...");
        Student s3 = new Student();
        s3.rollNo = 103;
        s3.name   = "Amit Verma";
        s3.branch = "Electronics";
        s3.age    = 20;
        s3.marks  = new double[]{35.0, 42.0, 38.0, 45.0, 30.0};
        s3.percentage = s3.calculatePercentage();
        s3.grade      = s3.calculateGrade();

        // ---- Calling displayDetails() ----
        System.out.println("\n========== Displaying All Students ==========");
        s1.displayDetails();
        System.out.println();
}}
```

Output

```
>>> Creating Student 1 using Parameterized Constructor...
>>> Creating Student 2 using Parameterized Constructor...
>>> Creating Student 3 using Default Constructor...

========== Displaying All Students ==========
??????????????????????????????????????????
?         STUDENT DETAILS                  ?
??????????????????????????????????????????
?  Roll No   : 101                         ?
?  Name      : Rahul Sharma                ?
?  Branch    : Computer Science            ?
?  Age       : 20                          ?
??????????????????????????????????????????
?            MARKS DETAILS                 ?
??????????????????????????????????????????
?  Maths     : 88.0                        ?
?  Science    : 92.0                        ?
?  English    : 78.0                        ?
?  History    : 85.0                        ?
?  CS         : 95.0                        ?
??????????????????????????????????????????
?  Percentage: 87.60                        ?
?  Grade     : A+ (Excellent)              ?
??????????????????????????????????????????
```

Fig 4.1 Output

Logical Applications:

1. What is the difference between a class and an object?

_____

_____

_____

2. What is the role of this keyword in a constructor?

_____

_____

_____

Date:

PRACTICAL - 5

AIM: Create Java programs using constructor, this and static keywords
a. Create a Java program to demonstrate default and parameterized constructors and the use of 'this' keyword.

Program/Implementation:

A: Constructors & this Keyword

```
public class BankAccount {

    // ===== FIELDS =====
    int    accountNo;
    String holderName;
    String accountType;
    double balance;
    String ifscCode;

    // Static field to auto-generate account numbers
    static int nextAccountNo = 1001;

    // ===== DEFAULT CONSTRUCTOR =====
    BankAccount() {
        this.accountNo   = nextAccountNo++;
        this.holderName  = "Unknown";
        this.accountType = "Savings";
        this.balance     = 0.0;
        this.ifscCode    = "BANK0000001";
        System.out.println("✓ Default Constructor called  → Account No: " + this.accountNo);
    }

    // ===== PARAMETERIZED CONSTRUCTOR 1 (name only) =====
    BankAccount(String holderName) {
        this();                      // Calls default constructor using 'this'
        this.holderName = holderName;    // Overrides default name
        System.out.println("✓ Constructor(name) called    → Holder: " + this.holderName);
    }

    // ===== PARAMETERIZED CONSTRUCTOR 2 (name + balance) =====
    BankAccount(String holderName, double balance) {
```

```
    this(holderName);               // Calls Constructor 1 using 'this'
    this.balance = balance;
    System.out.println("✓ Constructor(name,bal) called → Balance: " + this.balance);
}

// ===== PARAMETERIZED CONSTRUCTOR 3 (all fields) =====
BankAccount(String holderName, String accountType, double balance, String ifscCode) {
    this(holderName, balance);      // Calls Constructor 2 using 'this'
    this.accountType = accountType;
    this.ifscCode    = ifscCode;
    System.out.println("✓ Constructor(all) called     → Type: "  + this.accountType);
}

// ===== METHOD: Deposit using 'this' =====
BankAccount deposit(double amount) {
    if (amount > 0) {
        this.balance += amount;
        System.out.println("  [Deposit]  +" + amount
                + "  →  New Balance: " + this.balance);
    } else {
        System.out.println("  ✗ Invalid deposit amount!");
    }
    return this;   // Returns current object → enables method chaining
}

// ===== METHOD: Withdraw using 'this' =====
BankAccount withdraw(double amount) {
    if (amount > 0 && amount <= this.balance) {
        this.balance -= amount;
        System.out.println("  [Withdraw] -" + amount
                + "  →  New Balance: " + this.balance);
    } else {
        System.out.println("  ✗ Insufficient balance or invalid amount!");
    }
    return this;   // Returns current object → enables method chaining
}

// ===== METHOD: Display Account Details =====
void displayAccount() {
    System.out.println("┌──────────────────────────────────────┐");
    System.out.println("│         ACCOUNT DETAILS              │");
    System.out.println("└──────────────────────────────────────┘");
    System.out.printf ("  Account No   : %-20d %n", this.accountNo);
    System.out.printf ("  Holder Name  : %-20s %n", this.holderName);
```

```
        System.out.printf ("│  Account Type : %-20s│%n", this.accountType);
        System.out.printf ("│  Balance      : %-20.2f│%n", this.balance);
        System.out.printf ("│  IFSC Code    : %-20s│%n", this.ifscCode);
        System.out.println("└──────────────────────────────────────────────┘");
    }

    // ===== METHOD: Compare using 'this' =====
    void compareBalance(BankAccount other) {
        System.out.println("\n--- Balance Comparison ---");
        System.out.println("  " + this.holderName  + "  : ₹" + this.balance);
        System.out.println("  " + other.holderName + "  : ₹" + other.balance);
        if (this.balance > other.balance)
            System.out.println("  🏆 " + this.holderName + " has higher balance.");
        else if (this.balance < other.balance)
            System.out.println("  🏆 " + other.holderName + " has higher balance.");
        else
            System.out.println("  🏆 Both have equal balance.");
    }

    // ===== MAIN METHOD =====
    public static void main(String[] args) {


System.out.println("╔═══════════════════════════════════════════════╗");
        System.out.println("║  CONSTRUCTOR & this KEYWORD DEMO        ║");

System.out.println("╚═══════════════════════════════════════════════╝");

        // --- Default Constructor ---
        System.out.println("\n>>> Object 1: Default Constructor");
        BankAccount acc1 = new BankAccount();
        acc1.displayAccount();

        // --- Constructor with name only ---
        System.out.println("\n>>> Object 2: Constructor(name)");
        BankAccount acc2 = new BankAccount("Rahul Sharma");
        acc2.displayAccount();

        // --- Constructor with name + balance ---
        System.out.println("\n>>> Object 3: Constructor(name, balance)");
        BankAccount acc3 = new BankAccount("Priya Patel", 15000.00);
        acc3.displayAccount();
```

```
        // --- Full Parameterized Constructor ---
        System.out.println("\n>>> Object 4: Constructor(all fields)");
        BankAccount acc4 = new BankAccount("Amit Verma", "Current",
                                    50000.00, "BANK0001234");
        acc4.displayAccount();

        // --- 'this' for Method Chaining ---
        System.out.println("\n>>> Demonstrating Method Chaining using 'this'");
        System.out.println("Account: " + acc3.holderName);
        acc3.deposit(5000).deposit(3000).withdraw(2000).withdraw(500);

        System.out.println("\nAfter Chaining:");
        acc3.displayAccount();

        // --- 'this' for Object Comparison ---
        acc3.compareBalance(acc4);
    }
}
}
```

B: static Keyword

```
public class Counter {

    static int totalObjects = 0;
    static String bankName  = "Java National Bank";
    int    id;
    String ownerName;
    double balance;

    // ===== STATIC BLOCK =====
    // Runs ONCE when class is loaded into memory
    static {

System.out.println("╔═══════════════════════════════════════════════════════════════╗");
        System.out.println("║        static KEYWORD DEMO            ║");

System.out.println("╚═══════════════════════════════════════════════════════════════╝");
        System.out.println(">>> Static Block executed!");
        System.out.println("    Bank Name   : " + bankName);
        System.out.println("    System ready. Waiting for objects...\n");
    }

    // ===== CONSTRUCTOR =====
    Counter(String ownerName, double balance) {
```

```
    totalObjects++;              // Increment shared static counter
    this.id        = totalObjects;
    this.ownerName = ownerName;
    this.balance   = balance;
    System.out.println("  [+] Object " + this.id + " created → " + this.ownerName);
}

// ===== INSTANCE METHOD =====
// Can access both static and instance fields
void displayInfo() {
    System.out.printf("  ID: %-4d | Owner: %-15s | Balance: ₹%-10.2f | Bank: %s%n",
                      id, ownerName, balance, bankName);
}

// ===== STATIC METHOD =====
// Can ONLY access static fields — cannot use 'this'
static void displayBankInfo() {
    System.out.println("\n--- Bank Information ---");
    System.out.println("  Bank Name      : " + bankName);
    System.out.println("  Total Accounts : " + totalObjects);
    System.out.println("------------------------");
}

// ===== STATIC METHOD: Change Bank Name =====
static void setBankName(String name) {
    bankName = name;
    System.out.println("  ✓ Bank name updated to: " + bankName);
}

// ===== STATIC UTILITY METHOD (no object needed) =====
static double calculateInterest(double principal, double rate, int years) {
    return (principal * rate * years) / 100;
}

static double calculateCompoundInterest(double principal, double rate, int years) {
    return principal * Math.pow((1 + rate / 100), years) - principal;
}

// ===== MAIN METHOD =====
public static void main(String[] args) {

    // --- Static field accessed WITHOUT object ---
    System.out.println(">>> Accessing static field without object:");
    System.out.println("    Total Objects = " + Counter.totalObjects);
    System.out.println("    Bank Name     = " + Counter.bankName);
```

```
// --- Calling static method WITHOUT object ---
System.out.println("\n>>> Calling static method without object:");
Counter.displayBankInfo();

// --- Creating Objects ---
System.out.println(">>> Creating Accounts:");
Counter c1 = new Counter("Rahul Sharma",  25000.00);
Counter c2 = new Counter("Priya Patel",   42000.50);
Counter c3 = new Counter("Amit Verma",    15750.75);

// --- Static field reflects ALL objects ---
System.out.println("\n>>> Static field 'totalObjects' after creating 3 accounts:");
System.out.println("    Via Class  : Counter.totalObjects = " + Counter.totalObjects);
System.out.println("    Via Object : c1.totalObjects      = " + c1.totalObjects);
System.out.println("    (Both show same value — only ONE copy exists)\n");

// --- Display all accounts ---
System.out.println(">>> All Account Details:");
c1.displayInfo();
c2.displayInfo();
c3.displayInfo();

// --- Changing static field affects all objects ---
System.out.println("\n>>> Changing Bank Name via static method:");
Counter.setBankName("Bharat Digital Bank");
System.out.println("    After change — all accounts reflect new bank name:");
c1.displayInfo();
c2.displayInfo();
c3.displayInfo();

// --- Bank info after all objects ---
Counter.displayBankInfo();

// --- Static Utility Methods (no object needed) ---
System.out.println(">>> Static Utility Methods (Interest Calculator):");
double principal = 50000;
double rate      = 8.5;
int    years     = 3;

double si = Counter.calculateInterest(principal, rate, years);
double ci = Counter.calculateCompoundInterest(principal, rate, years);

System.out.println("  Principal : ₹" + principal);
System.out.println("  Rate      : "  + rate + "% per annum");
```

```
        System.out.println("  Years     : "  + years);
        System.out.printf ("  Simple Interest   : ₹%.2f%n", si);
        System.out.printf ("  Compound Interest : ₹%.2f%n", ci);

        // --- Summary of static vs instance ---

System.out.println("\n╔═══════════════════════════════════════════╗");
        System.out.println("║        static vs instance SUMMARY         ║");

System.out.println("╠═══════════════════════════════════════════╣");
        System.out.println("║  static field   → shared by ALL objects   ║");
        System.out.println("║  instance field → unique to EACH object   ║");
        System.out.println("║  static method  → called via Class name   ║");
        System.out.println("║  instance method→ called via object       ║");
        System.out.println("║  static block   → runs ONCE on class load ║");

System.out.println("╚═══════════════════════════════════════════╝");
    }
}
```

Output:

```
>>> Demonstrating Method Chaining using 'this'
Account: Priya Patel
  [Deposit]  +5000.0  ?  New Balance: 20000.0
  [Deposit]  +3000.0  ?  New Balance: 23000.0
  [Withdraw] -2000.0  ?  New Balance: 21000.0
  [Withdraw] -500.0  ?  New Balance: 20500.0

After Chaining:
??????????????????????????????????????????
?          ACCOUNT DETAILS            ?
??????????????????????????????????????????
?  Account No    : 1003                ?
?  Holder Name   : Priya Patel         ?
?  Account Type : Savings              ?
?  Balance       : 20500.00            ?
?  IFSC Code     : BANK0000001         ?
??????????????????????????????????????????

--- Balance Comparison ---
  Priya Patel   : ?20500.0
  Amit Verma   : ?50000.0
  ? Amit Verma has higher balance.
```

Fig 5.1 Output

```
--- Bank Information ---
  Bank Name      : Bharat Digital Bank
  Total Accounts : 3
-----------------------
>>> Static Utility Methods (Interest Calculator):
  Principal : ?50000.0
  Rate      : 8.5% per annum
  Years     : 3
  Simple Interest   : ?12750.00
  Compound Interest : ?13864.46

????????????????????????????????????????????????
?         static vs instance SUMMARY          ?
????????????????????????????????????????????????
?  static field   ? shared by ALL objects     ?
?  instance field ? unique to EACH object     ?
?  static method  ? called via Class name     ?
?  instance method? called via object         ?
?  static block   ? runs ONCE on class load   ?
????????????????????????????????????????????????
```

Fig 5.2 Output

Logical Applications:

1. What is constructor chaining and how is it achieved using this()?

_____

_____

_____

2. What is the difference between a static method and an instance method?

_____

_____

_____

Date:

PRACTICAL - 6

Aim: Create a Java program to demonstrate method overloading.

Program/Implementation:

```
public class MethodOverloading {

    // 1. Overloading by NUMBER of parameters
    int add(int a, int b)        { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }

    // 2. Overloading by TYPE of parameters
    double add(double a, double b) { return a + b; }
    String add(String a, String b) { return a + b; }

    public static void main(String[] args) {

        MethodOverloading obj = new MethodOverloading();

        // By number of parameters
        System.out.println("add(10, 20)       = " + obj.add(10, 20));
        System.out.println("add(10, 20, 30)   = " + obj.add(10, 20, 30));

        // By type of parameters
        System.out.println("add(10.5, 20.5)   = " + obj.add(10.5, 20.5));
        System.out.println("add(Hello, World) = " + obj.add("Hello ", "World"));
    }
}
```

OUTPUT:

```
add(10, 20)       = 30
add(10, 20, 30)   = 60
add(10.5, 20.5)   = 31.0
add(Hello, World) = Hello World
```

Fig 6.1 Output

Logical Applications:

1. Can two overloaded methods differ only in return type?

_____

_____

_____

2. What is the difference between method overloading and method overriding?

_____

_____

_____

Date:

PRACTICAL - 7

Aim*:* Create a Java program to perform various String operations of using built-in String methods.

Program/Implementation:

```
public class StringOperations {
   public static void main(String[] args) {

      String s1 = "Hello World";
      String s2 = "  Java Programming  ";
      String s3 = "hello world";

      // 1. Basic Info
      System.out.println("Original String    : " + s1);
      System.out.println("Length             : " + s1.length());
      System.out.println("charAt(4)          : " + s1.charAt(4));
      System.out.println("indexOf('o')       : " + s1.indexOf('o'));
      System.out.println("lastIndexOf('o')   : " + s1.lastIndexOf('o'));

      // 2. Case Conversion
      System.out.println("\ntoUpperCase()     : " + s1.toUpperCase());
      System.out.println("toLowerCase()      : " + s1.toLowerCase());

      // 3. Trimming & Replacing
      System.out.println("\nBefore trim        : \"" + s2 + "\"");
      System.out.println("After trim()       : \"" + s2.trim() + "\"");
      System.out.println("replace(l, r)      : " + s1.replace('l', 'r'));
      System.out.println("replaceAll(o, 0)   : " + s1.replaceAll("o", "0"));

      // 4. Substring & Splitting
      System.out.println("\nsubstring(6)       : " + s1.substring(6));
      System.out.println("substring(0, 5)    : " + s1.substring(0, 5));
      String[] words = s1.split(" ");
      System.out.print("split(\" \")         : ");
      for (String w : words) System.out.print("[" + w + "] ");
      System.out.println();

      // 5. Comparison
      System.out.println("\nequals(s3)         : " + s1.equals(s3));
      System.out.println("equalsIgnoreCase(s3): " + s1.equalsIgnoreCase(s3));
```

```
        System.out.println("compareTo(s3)       : " + s1.compareTo(s3));
        System.out.println("startsWith(Hello)   : " + s1.startsWith("Hello"));
        System.out.println("endsWith(World)     : " + s1.endsWith("World"));
        System.out.println("contains(World)     : " + s1.contains("World"));

        // 6. Conversion
        System.out.println("\nvalueOf(100)       : " + String.valueOf(100));
        System.out.println("valueOf(true)       : " + String.valueOf(true));
        System.out.println("toCharArray()       : ");
        for (char c : s1.toCharArray()) System.out.print(c + " ");
        System.out.println();

        // 7. Concatenation & isEmpty
        String s4 = s1.concat(" - Java");
        System.out.println("\nconcat()            : " + s4);
        System.out.println("isEmpty()           : " + s1.isEmpty());
        System.out.println("isEmpty(empty str)  : " + "".isEmpty());
    }
}
```

OUTPUT:

```
After trim()        : "Java Programming"
replace(l, r)       : Herro Worrd
replaceAll(o, 0)    : HellO WOrld

substring(6)        : World
substring(0, 5)     : Hello
split(" ")          : [Hello] [World]

equals(s3)          : false
equalsIgnoreCase(s3): true
compareTo(s3)       : -32
startsWith(Hello)   : true
endsWith(World)     : true
contains(World)     : true

valueOf(100)        : 100
valueOf(true)       : true
toCharArray()       :
H e l l o   W o r l d

concat()            : Hello World - Java
isEmpty()           : false
isEmpty(empty str)  : true
```

Fig 7.1 Output

Logical Applications:

1. What is the difference between equals() and == when comparing strings?

_____

_____

_____

2. What does trim() do and when is it useful?

_____

_____

_____

Date:

PRACTICAL - 8

Aim: Create a Java program to accept input using the Scanner class, store values and demonstrate autoboxing and unboxing

Program/Implementation:

```
import java.util.Scanner;
import java.util.ArrayList;

public class AutoboxingDemo {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      // ---- Accept Input ----
      System.out.print("Enter name    : ");
      String name = sc.nextLine();

      System.out.print("Enter age     : ");
      int age = sc.nextInt();

      System.out.print("Enter marks   : ");
      double marks = sc.nextDouble();

      System.out.print("Enter pincode : ");
      long pincode = sc.nextLong();

      // ---- Autoboxing: primitive → Wrapper ----
      Integer boxedAge     = age;      // int    → Integer
      Double  boxedMarks   = marks;    // double → Double
      Long    boxedPincode = pincode;  // long   → Long
      Boolean boxedPass    = marks >= 40; // boolean → Boolean

      System.out.println("\n--- Autoboxing (primitive → Wrapper) ---");
      System.out.println("int     → Integer : " + boxedAge);
      System.out.println("double  → Double  : " + boxedMarks);
      System.out.println("long    → Long    : " + boxedPincode);
      System.out.println("boolean → Boolean : " + boxedPass);

      // ---- Unboxing: Wrapper → primitive ----
      int    unboxedAge   = boxedAge;    // Integer → int
      double unboxedMarks = boxedMarks;  // Double  → double
```

```
        System.out.println("\n--- Unboxing (Wrapper → primitive) ---");
        System.out.println("Integer → int    : " + unboxedAge);
        System.out.println("Double  → double : " + unboxedMarks);

        // ---- Wrapper Utility Methods ----
        System.out.println("\n--- Wrapper Utility Methods ---");
        System.out.println("Integer.MAX_VALUE  : " + Integer.MAX_VALUE);
        System.out.println("Integer.MIN_VALUE  : " + Integer.MIN_VALUE);
        System.out.println("Integer.toBinary() : " + Integer.toBinaryString(age));
        System.out.println("Integer.toHex()    : " + Integer.toHexString(age));
        System.out.println("Double.parseDouble : " + Double.parseDouble("3.14"));
        System.out.println("Integer.parseInt   : " + Integer.parseInt("100"));

        // ---- ArrayList (requires Wrapper — shows autoboxing in collections) ----
        System.out.println("\n--- Autoboxing in ArrayList ---");
        ArrayList<Integer> marksList = new ArrayList<>();
        marksList.add(85);   // autoboxing
        marksList.add(90);
        marksList.add(78);
        marksList.add((int) marks); // explicit cast then autobox

        System.out.print("Marks List : ");
        int total = 0;
        for (int m : marksList) {   // unboxing in loop
            System.out.print(m + " ");
            total += m;             // unboxing for arithmetic
        }
        System.out.println("\nTotal      : " + total);
        System.out.println("Average    : " + (double) total / marksList.size());

        // ---- Summary ----
        System.out.println("\n--- Student Summary ---");
        System.out.println("Name    : " + name);
        System.out.println("Age     : " + unboxedAge);
        System.out.println("Marks   : " + unboxedMarks);
        System.out.println("Pincode : " + pincode);
        System.out.println("Result  : " + (boxedPass ? "PASS" : "FAIL"));

        sc.close();
    }
}
```

OUTPUT:

```
Enter name    : naman
Enter age     : 23
Enter marks   : 43
Enter pincode : 390024

--- Autoboxing (primitive ? Wrapper) ---
int     ? Integer : 23
double  ? Double  : 43.0
long    ? Long    : 390024
boolean ? Boolean : true

--- Unboxing (Wrapper ? primitive) ---
Integer ? int    : 23
Double  ? double : 43.0

--- Wrapper Utility Methods ---
Integer.MAX_VALUE  : 2147483647
Integer.MIN_VALUE  : -2147483648
Integer.toBinary() : 10111
Integer.toHex()    : 17
Double.parseDouble : 3.14
Integer.parseInt   : 100
```

Fig 8.1 Output

Logical Applications:

1. What is the difference between autoboxing and unboxing?

_____

_____

_____

2. Why does ArrayList require wrapper classes instead of primitive types?

_____

_____

_____

Date:

PRACTICAL - 9

Aim*:* Create a Java program to demonstrate single and multilevel inheritance.

Program/Implementation:

```
// ── Base Class ──
class Animal {
    String name;

    Animal(String name) {
        this.name = name;
        System.out.println("Animal: " + name);
    }

    void eat()   { System.out.println(name + " is eating."); }
    void sleep() { System.out.println(name + " is sleeping."); }
}

// ── Single Inheritance: Dog extends Animal ──
class Dog extends Animal {
    String breed;

    Dog(String name, String breed) {
        super(name);
        this.breed = breed;
        System.out.println("Dog breed: " + breed);
    }

    void bark() { System.out.println(name + " is barking."); }
}

// ── Multilevel Inheritance: Puppy extends Dog extends Animal ──
class Puppy extends Dog {
    int age;

    Puppy(String name, String breed, int age) {
        super(name, breed);
        this.age = age;
        System.out.println("Puppy age: " + age);
    }
```

```
    void weep() { System.out.println(name + " is weeping."); }
}

public class InheritanceDemo {
    public static void main(String[] args) {

        System.out.println("-- Single Inheritance --");
        Dog dog = new Dog("Bruno", "Labrador");
        dog.eat();
        dog.sleep();
        dog.bark();

        System.out.println("\n-- Multilevel Inheritance --");
        Puppy pup = new Puppy("Tommy", "Poodle", 2);
        pup.eat();
        pup.bark();
        pup.weep();
    }
}
```

OUTPUT:

```
-- Single Inheritance --
Animal: Bruno
Dog breed: Labrador
Bruno is eating.
Bruno is sleeping.
Bruno is barking.

-- Multilevel Inheritance --
Animal: Tommy
Dog breed: Poodle
Puppy age: 2
Tommy is eating.
Tommy is barking.
Tommy is weeping.
```

Fig 9.1 Output

Logical Applications:

1. What is the difference between single and multilevel inheritance?

_____

_____

_____

2. What is the role of super keyword in inheritance?

_____

_____

_____

Date:

PRACTICAL - 10

Aim: Write a Java practical to declare a 1D array of 10 integers, accept values from the user, and find the maximum, minimum, sum, and average of all elements.

Source Code:

```
import java.util.Scanner;

public class ArrayOperations {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      int[] arr = new int[10];

      System.out.println("Enter 10 integers:");
      for (int i = 0; i < 10; i++) {
         System.out.print("Element [" + i + "] : ");
         arr[i] = sc.nextInt();
      }

      int max = arr[0], min = arr[0], sum = 0;

      for (int i = 0; i < 10; i++) {
         if (arr[i] > max) max = arr[i];
         if (arr[i] < min) min = arr[i];
         sum += arr[i];
      }
      double avg = (double) sum / 10;
      System.out.println("\n-- Results --");
      System.out.println("Maximum : " + max);
      System.out.println("Minimum : " + min);
      System.out.println("Sum     : " + sum);
      System.out.println("Average : " + avg);
      sc.close();
   }
}
```

OUTPUT:

```
Enter 10 integers:
Element [0] : 1
Element [1] : 2
Element [2] : 3
Element [3] : 4
Element [4] :
6
Element [5] : 7
Element [6] : 8
Element [7] : 9
Element [8] : 12
Element [9] : 21

-- Results --
Maximum : 21
Minimum : 1
Sum     : 73
Average : 7.3
```

Fig 10.1 Output

Logical Applications:

1. Why is arr[0] used to initialize max and min instead of 0?

_____

_____

_____

2. Why is (double) cast needed when calculating the average of integers?

_____

_____

_____

Date:

PRACTICAL – 11

Aim: Write a Java practical to create a Student class with fields name, roll number, and marks using a parameterized constructor and a display() method to print details of multiple student objects.

Source Code:

```
public class Student {

   String name;
   int    rollNo;
   double marks;

   Student(String name, int rollNo, double marks) {
      this.name   = name;
      this.rollNo = rollNo;
      this.marks  = marks;
   }

   void display() {
      System.out.println("Roll No : " + rollNo);
      System.out.println("Name    : " + name);
      System.out.println("Marks   : " + marks);
      System.out.println("Grade   : " + (marks >= 75 ? "A" : marks >= 60 ? "B" : marks >= 40 ? "C"
: "F"));
      System.out.println("----------------------------");
   }

   public static void main(String[] args) {

      Student s1 = new Student("Rahul Sharma", 101, 88.5);
      Student s2 = new Student("Priya Patel",  102, 63.0);
      Student s3 = new Student("Amit Verma",   103, 35.0);

      System.out.println("-- Student Details --");
      s1.display();
      s2.display();
```

```
        s3.display();
    }
}
```

OUTPUT:

```
-- Student Details --
Roll No : 101
Name    : Rahul Sharma
Marks   : 88.5
Grade   : A
---------------------------
Roll No : 102
Name    : Priya Patel
Marks   : 63.0
Grade   : B
---------------------------
Roll No : 103
Name    : Amit Verma
Marks   : 35.0
Grade   : F
---------------------------
```

Fig 11.1 Output

Logical Applications:

1. What is the advantage of a parameterized constructor over a default constructor?

_____

_____

_____

2. Can a class have multiple constructors in Java?

_____

_____

_____

Date:

PRACTICAL – 12

Aim*:* Write a Java practical to demonstrate inheritance and method overriding by creating a parent class Animal and a child class Dog that overrides the sound() method using the super keyword.

Source Code:

```
class Animal {
   String name;
   Animal(String name) {
      this.name = name;
   }
   void sound() {
      System.out.println(name + " makes a generic sound.");
   }
   void eat() {
      System.out.println(name + " is eating.");
   }
}
class Dog extends Animal {
   String breed;
   Dog(String name, String breed) {
      super(name);
      this.breed = breed;
   }
   @Override
   void sound() {
      super.sound();                          // Calls parent method
      System.out.println(name + " barks: Woof Woof!"); // Overridden behavior
   }
}
public class OverridingDemo {
   public static void main(String[] args) {
      Animal a = new Animal("Animal");
      Dog    d = new Dog("Bruno", "Labrador");
      System.out.println("-- Parent Class --");
      a.sound();
```

```
        a.eat();
        System.out.println("\n-- Child Class (Overriding) --");
        d.sound();
        d.eat();
    }
}
```

OUTPUT:

```
-- Single Inheritance --
Animal: Bruno
Dog breed: Labrador
Bruno is eating.
Bruno is sleeping.
Bruno is barking.

-- Multilevel Inheritance --
Animal: Tommy
Dog breed: Poodle
Puppy age: 2
Tommy is eating.
Tommy is barking.
Tommy is weeping.
```

Fig 12.1 Output

Logical Applications:

1. What is the difference between @Override annotation and without it? wa

_____

_____

_____

2. What happens when super.sound() is called inside the child class?

_____

_____

_____

Date:

PRACTICAL – 13

Aim: Write a Java practical to demonstrate the Collections Framework by creating an ArrayList of student names to perform add, remove, and search operations and a HashSet to store and display unique roll numbers using a for-each loop.

Source Code:

```
import java.util.ArrayList;
import java.util.HashSet;

public class CollectionsDemo {
   public static void main(String[] args) {

      // ── ArrayList of Student Names ──
      ArrayList<String> students = new ArrayList<>();

      // Add
      students.add("Rahul Sharma");
      students.add("Priya Patel");
      students.add("Amit Verma");
      students.add("Neha Singh");

      System.out.println("-- ArrayList --");
      System.out.println("Students    : " + students);

      // Remove
      students.remove("Amit Verma");
      System.out.println("After Remove: " + students);

      // Search
      System.out.println("Contains Priya? : " + students.contains("Priya Patel"));
      System.out.println("Index of Neha   : " + students.indexOf("Neha Singh"));

      // For-each loop
      System.out.println("\nAll Students:");
      for (String s : students)
```

```
        System.out.println("  " + s);

    // ── HashSet of Roll Numbers ──
    HashSet<Integer> rollNos = new HashSet<>();

    rollNos.add(101);
    rollNos.add(102);
    rollNos.add(103);
    rollNos.add(101);   // duplicate — will be ignored
    rollNos.add(102);   // duplicate — will be ignored

    System.out.println("\n-- HashSet (unique roll numbers) --");
    System.out.println("Roll Numbers:");
    for (int r : rollNos)
        System.out.println("  " + r);

    System.out.println("Total Unique: " + rollNos.size());
  }
}
```

OUTPUT:

```
-- ArrayList --
Students    : [Rahul Sharma, Priya Patel, Amit Verma, Neha Singh]
After Remove: [Rahul Sharma, Priya Patel, Neha Singh]
Contains Priya? : true
Index of Neha   : 2

All Students:
  Rahul Sharma
  Priya Patel
  Neha Singh

-- HashSet (unique roll numbers) --
Roll Numbers:
  101
  102
  103
Total Unique: 3
```

Fig 13.1 Output

Logical Applications:

1. What is the difference between ArrayList and HashSet?

_____

_____

_____

2. Why does HashSet not allow duplicate values?

_____

_____

_____

Date:

PRACTICAL – 14

Aim: Write a Java practical to calculate the percentage of a student based on marks of 5 subjects and display the corresponding grade using if-else ladder control structure.

Source Code:

```
import java.util.Scanner;

public class StudentGrade {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      int[] marks = new int[5];
      String[] subjects = {"Maths", "Science", "English", "History", "CS"};

      System.out.println("Enter marks for 5 subjects:");
      for (int i = 0; i < 5; i++) {
         System.out.print(subjects[i] + " : ");
         marks[i] = sc.nextInt();
      }

      int sum = 0;
      for (int m : marks) sum += m;
      double percentage = sum / 5.0;

      String grade;
      if     (percentage >= 90) grade = "O - Outstanding";
      else if (percentage >= 75) grade = "A - Excellent";
      else if (percentage >= 60) grade = "B - Very Good";
      else if (percentage >= 50) grade = "C - Good";
      else if (percentage >= 40) grade = "D - Average";
      else                       grade = "F - Fail";

      System.out.println("\n-- Result --");
      System.out.println("Total      : " + sum + " / 500");
      System.out.printf ("Percentage : %.2f%%%n", percentage);
```

```
        System.out.println("Grade      : " + grade);
        sc.close();
    }
}
```

OUTPUT:

```
Enter marks for 5 subjects:
Maths : 21
Science : 22
English : 24
History : 25
CS : 54

-- Result --
Total      : 146 / 500
Percentage : 29.20%
Grade      : F - Fail

=== Code Execution Successful ===
```

Fig 14.1 Output

Logical Applications:

1. Why is sum / 5.0 used instead of sum / 5 for percentage?

_____

_____

_____

2. What is the difference between if-else ladder and switch statement?

_____

_____

_____